Maximizing the adjacent comparison classes of parking functions

Qiyuan Ma
Shanghai Jiao Tong University
mqyqwerty@sjtu.edu.cn
ORCID: 0009-0000-9532-0821

October 28, 2026
Research preprint, version 1.0.0

Abstract

A parking function of length n records each adjacent comparison by A, D, or T, according as it is a strict ascent, a strict descent, or a tie. We determine all comparison words with a largest class. For odd $n \geq 3$, the unique largest class has encoding $\mathrm{ADAD}\cdots\mathrm{AD}$. For even $n \geq 4$, the two alternating encodings tie for the maximum. At $n = 2$, all three one-letter classes have size one. The proof uses explicit injections that strictly eliminate tied candidates and repair the first pair of consecutive comparisons in the same direction. We also give an exact subset recurrence for the maximum and a reproducible collection of checks. This answers Problem 54 of Cruz et al. in the form of a self-contained research preprint; no independent expert peer review or historical-priority certification is claimed.

1 The maximizing words

A word $a = (a_1, \ldots, a_n) \in [n]^n$ is a parking function if its nondecreasing rearrangement $b_1 \leq \cdots \leq b_n$ satisfies $b_i \leq i$. Its comparison encoding $\mathrm{enc}(a) = w_1 \cdots w_{n-1}$ is defined by

$$w_i = \mathrm{A} \quad (a_i < a_{i+1}), \qquad w_i = \mathrm{D} \quad (a_i > a_{i+1}), \qquad w_i = \mathrm{T} \quad (a_i = a_{i+1}).$$

Write $F_n(w) = |\{a \in \mathrm{PF}_n : \mathrm{enc}(a) = w\}|$. Let $Z_n^{\mathrm{A}} = \mathrm{ADAD}\cdots$ and $Z_n^{\mathrm{D}} = \mathrm{DADA}\cdots$, both of length $n-1$, and set $Z_1^{\mathrm{A}} = \varepsilon$.

Theorem 1.1. *All maximizing encodings are given by*

$$\operatorname*{arg\,max}_{w \in \{\mathrm{A},\mathrm{D},\mathrm{T}\}^{n-1}} F_n(w) = \begin{cases} \{\varepsilon\}, & n = 1, \\ \{\mathrm{A}, \mathrm{D}, \mathrm{T}\}, & n = 2, \\ \{Z_n^{\mathrm{A}}\}, & n \geq 3 \text{ odd}, \\ \{Z_n^{\mathrm{A}}, Z_n^{\mathrm{D}}\}, & n \geq 4 \text{ even}. \end{cases}$$

Consequently the maximum is $M_n = F_n(Z_n^{\mathrm{A}})$ for every $n \geq 1$. The recurrence in Section 7 computes each M_n without any remaining maximization.

Cruz et al. [1] ask which full statistic encoding has the largest class. The analogous extremal result for ordinary permutation descent classes is classical; see, for example, Sagan–Yeh–Ziegler [2]. Repeated values prevent an immediate transfer of a multiset-preserving permutation argument. The alternating parking-function counts themselves were recorded in 2015 as OEIS entries A260694 and A264963 [3, 4]. We do not claim these numerical sequences as new. Recent recurrences for aggregate peak and tie statistics [5] concern a different refinement from a fixed complete comparison word.

2 Closure and rank assignments

For a positive-integer word a define $C_a(t) = |\{i : a_i \le t\}|$. The parking condition is equivalent to

$$C_a(t) \ge t \qquad (1 \le t \le n).$$

Lemma 2.1. *Reordering a parking function preserves the parking condition. Replacing one entry by a smaller positive integer also preserves it. If a parking function contains at least two copies of x, increasing one copy to $x+1$ preserves the parking condition.*

Proof. Reordering leaves every $C_a(t)$ unchanged, and decreasing an entry can only increase these counts. For the last statement,

$$C_a(x) \ge C_a(x-1) + 2 \ge x + 1,$$

where $C_a(0) = 0$. The modification decreases only $C_a(x)$, by one; all other integer-threshold counts are unchanged. Thus the inequalities still hold. In particular, the modification cannot produce a value larger than n in a parking function. □

Collapse each maximal run of ties of a comparison word into one block. The blocks form a directed path, with an arrow from the smaller block to the larger block. Its transitive closure is a poset. Assigning the distinct values $1, \dots, m$ to its m blocks in a linear extension gives a parking function: each of the preceding $j-1$ block values occurs at least once before the first occurrence of value j in sorted order. Thus every occurrence of j has sorted position at least j. We use this rank-assignment observation also after merging nonadjacent minimal vertices or nonadjacent maximal vertices. Such a merge does not create a directed cycle.

3 The two odd alternating directions

The following map will also be used on prefixes.

Lemma 3.1. *Let $k \ge 3$ be odd, and suppose $a_1 > a_2 < a_3 > \cdots < a_k$. Define*

$$R(a) = \begin{cases} (a_2, \dots, a_k, a_1), & a_1 < a_k, \\ (a_k, a_1, \dots, a_{k-1}), & a_1 > a_k, \\ (a_2, \dots, a_{k-1}, a_1, a_2), & a_1 = a_k. \end{cases}$$

Then R is an injection into the alternating words with low endpoints. Its last entry is strictly smaller than a_k. Its multiset is unchanged, or one value is decreased.

Proof. The adjacent inequalities in each displayed tuple give the required alternation. In the first two cases the last entries are $a_1 < a_k$ and $a_{k-1} < a_k$. In the last case it is $a_2 < a_1 = a_k$; one copy of a_1 is replaced by the smaller value a_2. If the output is b, its endpoints satisfy respectively $b_1 < b_k$, $b_1 > b_k$, and $b_1 = b_k$. Thus the branch is recognizable. The first two inverse maps are the opposite cyclic shifts; the third inverse is

$$(b_{k-1}, b_1, \dots, b_{k-2}, b_{k-1}).$$

□

Proposition 3.2. *For odd $n \ge 3$, $F_n(Z_n^{\mathrm{D}}) < F_n(Z_n^{\mathrm{A}})$. For even n, the two alternating classes have equal size.*

Proof. Lemma 3.1 and Lemma 2.1 give the injection for odd n. It is not onto. The word

$$b = (1,2)^{(n-3)/2}(1,n,1)$$

is a parking function with encoding Z_n^{A}; juxtaposition here denotes concatenation. Its endpoints agree. Its only possible inverse under R contains two copies of n, so is not a parking function. For even n, reversing all coordinates interchanges the two alternating encodings and preserves the parking condition. □

4 Strict repair of the first defect

For a binary comparison word $w \in \{\mathrm{A},\mathrm{D}\}^{n-1}$, call $w_i = w_{i+1}$ a defect. Suppose w has a defect, and let $k \in \{2,\dots,n-1\}$ be the smallest index with $w_{k-1} = w_k = c$. The first $k-1$ comparisons are alternating. Let w^+ interchange A and D in these first $k-1$ positions and leave the other positions unchanged.

Theorem 4.1. *For this first-defect operation,* $F_n(w) < F_n(w^+)$.

Set $s = (a_1,\dots,a_k)$ and $t = a_{k+1} \geq 1$. All maps below leave $a_{k+1},\dots,a_n$ fixed. Put

$$\mathrm{rev}(s) = (a_k,\dots,a_1), \qquad \mathrm{irev}(s) = (a_1, a_{k-1}, a_{k-2},\dots,a_2,a_k),$$

where the second map reverses only the interior and is the identity when $k = 2$.

4.1 Even prefixes

If k is even and $c = \mathrm{A}$, the prefix starts low and ends high, with $a_k < t$. Define

$$E_{\mathrm{A}}(s;t) = \begin{cases} \mathrm{rev}(s), & a_1 < t, \\ \mathrm{irev}(s), & a_1 \geq t. \end{cases}$$

The first branch reverses the alternating direction and has new last entry $a_1 < t$. For the second branch,

$$a_1 \geq t > a_k > a_{k-1}, \qquad a_2 > a_1 \geq t > a_k.$$

These inequalities check the new endpoint comparisons; the interior comparisons reverse by alternation. The last entry is still $a_k < t$. Thus both branches give the required encoding and preserve the comparison with t. Their first output entries are respectively less than t and at least t. This distinguishes the branches; the same reversal in each branch recovers the input.

If k is even and $c = \mathrm{D}$, use

$$E_{\mathrm{D}}(s;t) = \begin{cases} \mathrm{rev}(s), & a_1 > t, \\ \mathrm{irev}(s), & a_1 \leq t. \end{cases}$$

Now $a_k > t$. In the second branch

$$a_1 \leq t < a_k < a_{k-1}, \qquad a_2 < a_1 \leq t < a_k,$$

so the new endpoint comparisons are correct and the last entry remains greater than t. The first output entries are respectively greater than t and at most t, giving the inverse branches. All these operations merely reorder the prefix, hence preserve the parking condition.

4.2 Odd prefixes

If k is odd and $c = \mathrm{A}$, the prefix has high endpoints and $a_k < t$. Apply R from Lemma 3.1. It reverses the alternating direction and decreases the last entry, preserving the following ascent and the parking condition.

For the remaining case we need an upper-threshold version.

Lemma 4.2. *Let $k \geq 3$ be odd, let a be alternating with low endpoints, and suppose $a_k > t \geq 1$. Then*

$$S_t(a) = \begin{cases} (a_k, a_1, \ldots, a_{k-1}), & a_1 < a_k, \\ (a_2, \ldots, a_k, a_1), & a_1 > a_k, \\ (a_2, \ldots, a_{k-1}, t, a_k), & a_1 = a_k \end{cases}$$

is an injection into high-endpoint alternating words, with last entry at least $a_k > t$. It preserves the multiset or decreases one value.

Proof. The first two branches are cyclic shifts; their last entries are strictly greater than a_k. In the last branch write $x = a_1 = a_k > t$. The inequalities $a_2 > x$ and $a_{k-1} > x > t$ give the required high-endpoint alternation; one copy of x is replaced by the positive value t, and the last entry remains x.

Every output b has unequal endpoints, so $R(b)$ uses only a cyclic shift. On the first two branches $R(b) = a$, whose last entry is greater than t. On the last branch

$$R(b) = (x, a_2, \ldots, a_{k-1}, t)$$

ends in exactly t. This distinguishes the last branch from the others. Its inverse is $(b_k, b_1, \ldots, b_{k-2}, b_k)$; the other inverse is $R(b)$. □

If k is odd and $c = \mathrm{D}$, the prefix has low endpoints and $a_k > t$. Apply S_t to it. This preserves the following descent, the required target encoding, and the parking condition by Lemma 2.1. We have constructed injections in all four cases.

4.3 Strictness in all four cases

We exhibit a target parking function outside the image in each case. All the target encodings here are binary.

For even k and $c = \mathrm{A}$, the first vertex of the target prefix is maximal in the full comparison poset, and its kth vertex is minimal. Give them values n and 1, and give the other vertices the distinct values $2, \ldots, n-1$ in a linear extension. This permutation is a parking function. Its first entry is at least t, so its only possible inverse reverses the interior and retains first entry n. The inverse cannot start with an ascent. It is not a source word. For even k and $c = \mathrm{D}$, give the target's first vertex value 1 and its kth vertex value n, and rank the remaining vertices. Its only possible inverse retains first entry 1 and cannot start with a descent. This also covers $k = 2$.

For odd k and $c = \mathrm{A}$, the target prefix has low endpoints, both minimal in the full comparison poset. Merge these endpoints as one block of value 1, give the last prefix peak (vertex $k-1$) value n, and give the other vertices the distinct ranks $2, \ldots, n-2$. The sorted word is $(1, 1, 2, \ldots, n-2, n)$, hence is a parking function. Its only possible inverse under R contains two copies of n.

For odd k and $c = \mathrm{D}$, the target prefix has high endpoints, both maximal in the full comparison poset. Merge them and rank the resulting $n-1$ blocks. This gives a target parking function whose prefix endpoints agree. All outputs of S_t have unequal prefix endpoints. This proves Theorem 4.1.

Corollary 4.3. *Every nonalternating binary encoding is strictly smaller than some alternating encoding.*

Proof. Write $\delta(w) = |\{i : w_i = w_{i+1}\}|$. Flipping the first $k-1$ letters preserves all comparisons between letters within that prefix, fixes the suffix, and changes the boundary pair $w_{k-1} = w_k$ to an unequal pair. Thus $\delta(w^+) = \delta(w) - 1$. Iterating Theorem 4.1 terminates at an alternating word. □

5 Strict elimination of ties

Theorem 5.1. *For $n \geq 3$, every encoding containing* T *is strictly smaller than some binary encoding of the same length.*

Proof. First construct non-strict injections which remove one tie at a time. Choose the rightmost T. If it is the last comparison, change the right member of the pair (x, x) to $x+1$, changing T to A. If its next letter is D, the local triple is (x, x, z) with $x > z$; use $(x, x+1, z)$, changing TD to AD. Lemma 2.1 applies and subtracting the increment is an inverse.

Otherwise the next comparison is A. Let q be the longest alternating segment starting there, and let $s = (s_1, \ldots, s_L)$ have encoding Tq, with $s_1 = s_2 = x$. Any comparison after this segment is the same as the last letter of q, because no tie lies to its right. The following maps keep the first entry x fixed.

If q ends in A, so L is odd, use

$$\Phi(s) = (x, s_3, \ldots, s_L, \min(x, s_{L-1})).$$

The encoding is qD. The new last entry is smaller than s_L, so any following ascent remains valid. One copy of x is replaced by a value at most x. If b is the output, the inverse is $(b_1, b_1, b_2, \ldots, b_{L-1})$.

If q ends in D, so L is even, put $u = s_L$ and use

$$\Psi(s) = \begin{cases} (x, s_3, \ldots, s_L, x), & u < x, \\ (x, u, x, s_3, \ldots, s_{L-1}), & u > x, \\ (x, s_3, \ldots, s_{L-1}, x, x+1), & u = x. \end{cases}$$

The encoding is qA. The new last entries are respectively $x > u$, $s_{L-1} > u$, and $x + 1 > u$, preserving any following descent. The first two cases reorder; in the third, the input contains three copies of x and one is increased to $x+1$, as allowed by Lemma 2.1. The output branches are distinguished by

$$\begin{array}{l|l} u < x & b_L = b_1 = x \\ u > x & b_L > x, \quad b_2 < b_L \\ u = x & b_L > x, \quad b_2 \geq b_L. \end{array}$$

The corresponding inverse tuples are

$$\begin{aligned} &(x, x, b_2, \ldots, b_{L-1}), \\ &(x, x, b_4, \ldots, b_L, b_2), \\ &(x, x, b_2, \ldots, b_{L-2}, x). \end{aligned}$$

All outside comparisons have been preserved. The target encoding depends only on the source encoding, and exactly one T disappears. Iteration therefore gives an injection to a fixed binary class.

We now check strictness. When the selected tie is followed by A, every image under Φ or Ψ has a repeated value within the new strictly alternating segment. Rank the target's tie blocks distinctly as in Section 2. The resulting parking function has distinct values within that segment and is missing from the image, so this step is strict.

If the selected tie is followed by D, the image of the simple increment has difference exactly 1 on its newly created ascent. The target has a singleton peak at that ascent. Give this peak value n. If the target has $m < n$ tie blocks, give the remaining blocks the ranks $1, \dots, m-1$, so its left neighbor is at most $n-2$. If $m = n$, deleting the peak separates a nonempty left part from a nonempty right part. Take a maximal element of the right part last among the other vertices, with rank $n-1$. Again the peak's left neighbor is at most $n-2$. These assignments are parking functions and have a gap at least 2, so this step is strict.

It remains to consider a rightmost tie at the end. Apply the terminal increment first. If another tie remains, its next letter is A or D, and the next step is strict by the preceding arguments. If there was just one tie, its preceding letter exists when $n \geq 3$. If that letter is D, the target ends in DA. Assign its last valley value 1, its final entry value n, and rank the other vertices in between. The last gap is $n-1 > 1$, so this target is not an image of the increment. If the preceding letter is A, the first target ends in AA. Swap its last two entries, giving an injection to a word ending in AD. Its image additionally requires the first of those last three entries to be smaller than the last entry. Give the new peak value n, the last entry value 1, and rank the other vertices in between to obtain a missing target. Thus a strict comparison is obtained in every case, proving the theorem. □

6 Conclusion of the classification

For $n \geq 3$, Theorem 5.1 excludes every tied word. Corollary 4.3 excludes every nonalternating binary word. Proposition 3.2 selects Z_n^{A} for odd n and gives equality between the two alternating words for even n. All exclusions are strict. At $n = 1$ there is only (1), and at $n = 2$ the parking functions are $(1,1), (1,2), (2,1)$. This proves Theorem 1.1, including all equality cases.

7 An exact recurrence for the maximum

Let Q_n be the poset on $V = [n]$ with

$$1 <_{Q_n} 2 >_{Q_n} 3 <_{Q_n} 4 >_{Q_n} \cdots .$$

For a set S of already assigned positions, let

$$\mathcal{M}(S) = \{i \notin S : i \text{ odd}\} \ \cup \ \{i \notin S : i \text{ even and all adjacent odd positions lie in } S\}.$$

Initialize $d_0(\varnothing) = 1$ and $d_0(S) = 0$ otherwise. For $j = 1, \dots, n$, start a zero layer and perform, for every S and every $R \subseteq \mathcal{M}(S)$,

$$d_j(S \cup R) \mathrel{+}= d_{j-1}(S) \qquad \text{if } |S \cup R| \geq j.$$

Empty R is allowed. Then

$$\boxed{M_n = d_n([n]).}$$

Indeed, S records the positions with value at most $j-1$, and R records exactly those with value j. Strict order constraints permit precisely subsets of the remaining minimal elements. The cumulative-size inequality is exactly the parking condition. Every permissible assignment is counted once. An on-the-fly implementation has a crude time bound $O(n3^n)$ and two-layer space bound $O(2^n)$; the supplied implementation caches transitions and uses arbitrary-precision integers.

n	M_n	n	M_n
1	1	7	9470
2	1	8	104436
3	4	9	1432713
4	17	10	20709209
5	120	11	354493902
6	915	12	6343919118

8 Executable checks and provenance

The repository provides the displayed injections, inverse maps, missing-image witnesses, full ternary encoding counts through $n = 10$, and maximum values through $n = 20$. All 18 248 parking functions of lengths at most six were passed through the complete composed injection and recovered by its inverse, with injectivity checked separately for every source encoding. Direct enumeration, weighted ordered set partitions, and poset assignment agree on every class through $n = 6$. Every nonalternating binary encoding through $n = 10$ was checked against the strict first-defect inequality and a constructed missing-image witness. These finite tests audit the implementation; the proofs above do not depend on an enumeration cutoff.

AI assistance. OpenAI Codex assisted extensively with mathematical exploration, proof development, programming, checking, literature screening, and English exposition. This is an AI-assisted research preprint. Independent expert peer review and proof-assistant formalisation are not claimed. See the repository's `AI_ASSISTANCE.md` and `VERIFICATION.md` for the released scope.

Availability and licensing. The manuscript, documentation, and JSON data are licensed under CC BY 4.0. The Python and C++ code is licensed under MIT. The repository is https://github.com/KevinMaSAI/parking-function-encoding-maxima.

References

[1] A. Cruz, P. E. Harris, K. J. Harry, J. Kretschmann, M. McClinton, A. Moon, J. O. Museus, and E. Redmon, *On Some Discrete Statistics of Parking Functions*, Journal of Integer Sequences **27** (2024), Article 24.8.6. https://cs.uwaterloo.ca/journals/JIS/VOL27/Harris/harris13.pdf.

[2] B. E. Sagan, Y.-N. Yeh, and G. M. Ziegler, *Maximizing Möbius functions on subsets of Boolean algebras*, Discrete Mathematics **126** (1994), 293–311. https://doi.org/10.1016/0012-365X(94)90273-9.

[3] The OEIS Foundation Inc., *A260694: Number of up-down parking functions of length n*, entry initiated in 2015. https://oeis.org/A260694.

[4] The OEIS Foundation Inc., *A264963: Number of down-up parking functions of length n*, entry initiated in 2015. https://oeis.org/A264963.

[5] P. Alexandersson, *Parking functions, Smirnov words, and noncrossing Chow polynomials*, arXiv:2609.05131v1 (2026). https://arxiv.org/abs/2609.05131.